

TTFL: Trust-Flow-Driven Trusted Federated Learning for Edge Computing

Jiakai Zhou, Chao Bu, Shuo Zhang, Xinyu Liu

School of Computer Science and Engineering

Tianjin University of Technology, Tianjin, China

Email: wllsp0@stud.tjut.edu.cn, bc_0722@163.com

Abstract—Federated learning (FL) enables collaborative edge intelligence without moving raw data, but open participation creates a trust-continuity problem: a client may execute an admissible training process while producing a harmful update, and a benign long-tail client may look anomalous under Non-IID data. This paper presents TTFL as a trust-flow framework that connects hardware-anchored admission, runtime evidence, dual-stream reputation evolution, and layer-wise aggregation. The central design separates long-term utility from short-term attack risk: historical contribution supports benign-client retention, while the risk stream captures harmful behavior that a favorable history should not conceal. Layer-wise gating, clipping, and survivor weight re-normalization translate these signals into differentiated aggregation privileges. Experiments on Flower/PyTorch with CIFAR-10, 20 admitted clients, six mixed attack types, and five repeated runs show 92.31% accuracy and 10.21% ASR under a 30% malicious setting, with no permanent false ban in the tested runs.

Index Terms—Federated learning, edge computing, trusted execution environment, risk modeling, layered aggregation, backdoor defense

1. Introduction

FL enables edge devices to train a shared model through parameter exchange while keeping their raw samples local [1], [2]. In an open edge deployment, however, the server must assess both the execution that produced an update and its effect on the global model. This assessment becomes difficult under Non-IID data: a legitimate long-tail client may repeatedly deviate from a reference direction, whereas an attacker may build a favorable record before submitting a harmful update. A single reputation score makes these cases difficult to separate. Tightening its threshold can remove useful heterogeneous contributions, while allowing past performance to compensate for current anomalies can delay the response to an attack. The resulting challenge is to preserve evidence of long-term contribution without allowing that evidence to conceal short-term risk.

Existing defenses use geometric, statistical, reference-based, or temporal evidence to flag unusual updates [3], [4], [5], [6], [7]. Backdoor-oriented methods additionally examine model behavior, learned representations, clusters, and

collusion [8], [9], [10], [11]. Such tests say little about how local training was executed, and the same tests can interpret Non-IID drift as an attack. TEE-assisted FL contributes a hardware-rooted execution boundary [12], [13], [14], [15]; admission based on that boundary still leaves the semantic safety of an admitted update unresolved.

The proposed *Trust Flow* policy addresses this tension by maintaining historical utility and short-term risk as separate states. Historical utility governs how a client retains or recovers its contribution, while current risk can restrict an update despite a favorable earlier record. These states connect execution admission, content auditing, and layer-wise aggregation: Intel TDX establishes the execution boundary, current evidence determines the submitted update’s influence, and persistence rules govern subsequent participation. Assigning evidence to these different decisions allows the framework to distinguish a temporarily unhelpful client from one that presents sustained risk. The contributions are:

- 1) A dual-stream state representation gives historical contribution and short-term risk separate roles, preserving benign-client retention while allowing a rapid response to attacks.
- 2) A state-to-aggregation policy links recoverable suspicion, persistent-risk escalation, and layer-specific weighting. Content quality can lower a contribution without, by itself, forcing irreversible exclusion.
- 3) An empirical analysis relates the single-stream and HistPerf-only ablations to the retention–detection tradeoff, and separates the effects of risk probes from those of weight re-normalization.

2. Background and Related Work

Federated learning in edge computing. Local training and server aggregation allow clients to collaborate without transferring raw samples, but leave the server dependent on the evidence accompanying each update. Heterogeneous data distributions complicate that assessment because a useful long-tail contribution may differ from the majority. Execution provenance and update quality consequently serve different purposes: the former establishes how an update was produced, while the latter determines how it should contribute to learning.

Poisoning and backdoor attacks. Sign flipping and gradient scaling disrupt optimization by changing update directions or magnitudes. Trigger, clean-label, and semantic backdoors instead seek targeted errors while preserving plausible main-task behavior, so their updates can remain close to an expected direction [16], [17]. Across rounds, an attacker may also accumulate a favorable history before injecting a harmful update, making current behavior and past contribution potentially conflicting sources of evidence.

Active defenses and their limits. Krum and Trimmed Mean remove geometric outliers [3], [4], FLTrust weights updates against a clean direction [5], and FoolsGold and FLDetector use cross-client or cross-round evidence [6], [7]. DeepSight, FLAME, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA inspect representations, backdoor behavior, collusion, or privacy-preserving updates [8], [9], [10], [11], [18], [19], [20]. TMT-FL and related systems connect execution integrity to model training [15]. These approaches observe different evidence spaces. Trust Flow places an interface between them: attestation determines execution eligibility, privacy-aware checks expose bounded statistics, content probes inspect the current update, and the two temporal streams decide future privileges. One observation can therefore alter a layer weight, a recoverable state, or a persistent state only when the corresponding rule is satisfied. The experiments use classical methods as matched aggregation baselines and stateful variants to isolate this temporal interface.

TEE-assisted trusted training. TEEs protect execution and key material and have been used to support FL integrity and attack mitigation [12], [13], [14], [15], [21]. A quote authenticates the measured path, freshness, and update provenance. TTFL combines that hardware evidence with privacy-aware verification and cloud-side semantic auditing before assigning aggregation privileges.

3. System Model and Threat Assumptions

Figure 1 presents the client-edge-cloud environment, consisting of a central aggregation server S and K edge clients $C = \{c_1, \dots, c_K\}$. In round t , admitted clients train from the distributed model $W^{(t-1)}$ and submit updates $\Delta W_k^{(t)}$ for auditing and aggregation. The server then computes $W^{(t)} = W^{(t-1)} + \text{Agg}(\{\Delta W_k^{(t)}\})$, with the global learning objective

$$\min_W \mathcal{L}(W) = \sum_{k=1}^K p_k \mathcal{L}_k(W), \quad p_k = |\mathcal{D}_k| / \sum_j |\mathcal{D}_j|. \quad (1)$$

The client side performs local training and supplies an attestation-bound report, while privacy-aware verification exposes the declared data and runtime statistics needed for admission and audit. Raw samples remain local. Using the submitted update and this evidence, the cloud server evolves the client’s trust states and determines its aggregation privilege. The division of responsibilities connects execution verification to model-level control: admission establishes

the provenance of an update, and subsequent auditing determines its contribution. Because these decisions require individual updates and audit evidence, the workflow does not use cryptographic secure aggregation to hide them from the server.

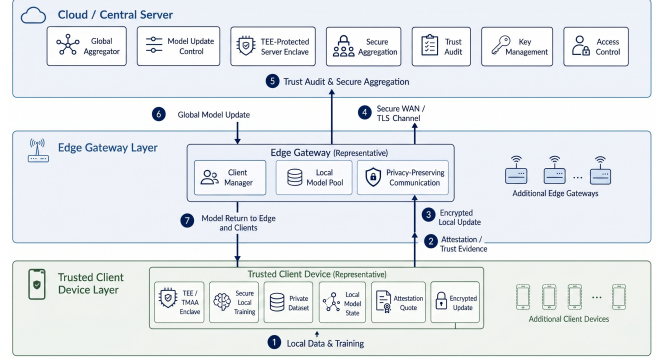


Figure 1. TEE-anchored client-edge-cloud architecture.

Threat model. Figure 2 summarizes the attack surface. The server is honest-but-curious and follows the aggregation protocol. It can see individual updates and audit statistics, but it does not change the stated state-transition rules. An admitted client may alter labels, samples, or optimization objectives and may conduct label flipping, trigger backdoors, clean-label poisoning, semantic backdoors, sign flipping, or gradient scaling.

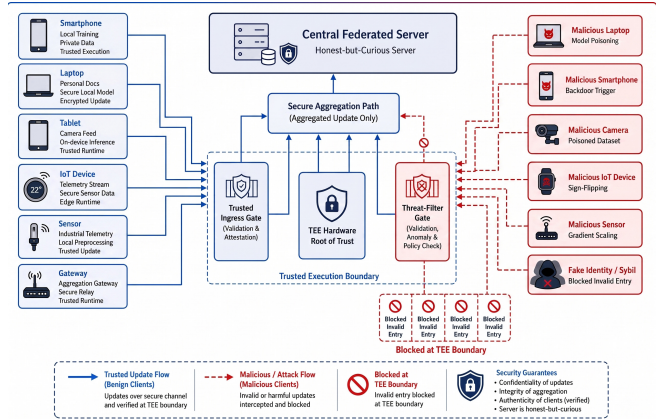


Figure 2. Composite poisoning and penetration threat model.

The client execution path uses Intel TDX. The TEE protects attestation-critical code and keys against a compromised host, while privacy-aware verification and cloud-side auditing examine selected statistics and model-update behavior without exposing raw samples. Passing execution admission does not bypass content auditing. Physical probing and advanced microarchitectural side-channel attacks are excluded. The assumed malicious fraction is below 50%; the 50% experiment is a boundary stress test rather than evidence of a theoretical Byzantine guarantee. A small clean validation gradient can guide content auditing, and the

trusted-peer branch supplies a reference direction when that gradient is unavailable.

Design goals. Trust Flow is designed to maintain convergence under the stated attacks, avoid permanent false bans of long-tail clients, and keep the assurance boundary explicit. Execution, current content, historical utility, and recent risk retain separate meanings until layer-wise aggregation. The reported TrustReport size is about 15 KB, and server aggregation plus audit increases from 2.10 s to 4.85 s. The auxiliary security path measures approximately 11 ms per admitted-client round, while device-side TEE Quote generation, signing, and lightweight monitoring measure approximately 12–15 ms per admitted-client round. These component measurements are reported separately from server processing and end-to-end communication.

4. Trust-Flow Framework

Each FL round passes through four linked decision stages: admission and runtime assessment, reference-based content auditing, dual-stream state evolution, and risk-gated layered aggregation. Global model update and feedback close the five-phase loop shown in Fig. 3.

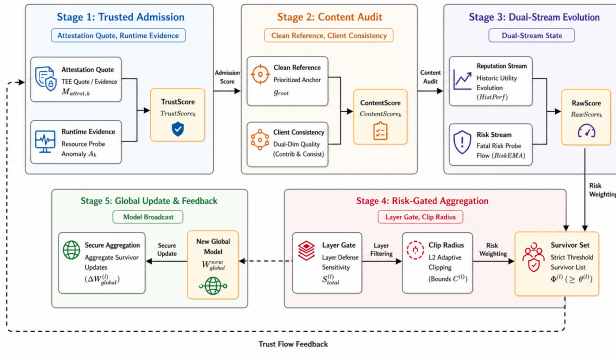


Figure 3. Overview of the five-phase trust-flow defense loop.

The stages in Fig. 3 pass progressively different evidence to the aggregation decision. Admission and runtime assessment produce $TrustScore$, content auditing produces $ContentScore$, and state evolution accumulates evidence in $HistPerf$ and $RiskEMA$. Their combination yields $RawScore$, which controls layer privileges. The server first removes clients that fail the admission proof, then scores the remaining updates against the reference and peer group. Keeping contribution and risk distinct allows a weak single-round contribution to reduce influence without immediately ending a benign client’s opportunity to recover.

4.1. Admission and Content Auditing

Each admitted client passes a TEE admission check. Let $M_{attest,k} \in \{0, 1\}$ indicate whether its attestation report is valid. During initialization, the client uses protected key material to generate a remote-attestation quote covering code

identity and runtime configuration; communication privileges follow a successful check. During local training, the trusted management agent monitors behavioral sequences $\mathcal{X}_k = \{\Delta grad, \Delta loss, instr_dist\}$. Their entropy is mapped to measurement uncertainty:

$$H(\mathcal{X}_k) = - \sum_i p(x_i) \log p(x_i), \quad (2)$$

$$R_k^{(t)} = \exp\left(\lambda H(\mathcal{X}_k^{(t)})\right).$$

High-entropy behavior is treated as uncertain evidence rather than a direct maliciousness label. Following the filtering view, the trust estimate and privilege factor are

$$\hat{T}_k^{(t)} = \hat{T}_k^{(t-1)} + K_k^{(t)}(Z_k^{(t)} - \hat{T}_k^{(t-1)}), \quad (3)$$

$$TrustScore_k^{(t)} = \hat{T}_k^{(t)}(1 + P_k^{(t)})^{-1}.$$

Here $Z_k^{(t)}$ is the observation, $K_k^{(t)}$ the Kalman gain, and $P_k^{(t)}$ the covariance. High entropy reduces the gain and limits the influence of an uncertain observation on the cross-round trust estimate. The covariance discount also reduces privilege when confidence is low. Admission therefore supplies a continuously updated factor for subsequent auditing and aggregation.

The server next constructs a clean guiding direction. When a small clean validation gradient is available, $g_{root} = g_{root_clean}$. Otherwise, admitted clients with high trust and low previous risk supply the reference, with weights $\omega_k^{ref} = TrustScore_k(1 - RiskEMA_k^{prev})^2$. The clean reference provides a starting direction, while peer consistency and temporal states contribute additional evidence. This fallback keeps reference construction available when a server-side clean set is small or absent. For each update, content quality combines reference consistency and peer consistency:

$$S_{contrib,k} = \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, g_{root})), \quad (4)$$

$$S_{consist,k} = \frac{1}{\sum_{j \neq k} TrustScore_j + \varepsilon} \times \sum_{j \neq k} \left[TrustScore_j \times \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, \Delta W_j)) \right], \quad (5)$$

$$ContentScore_k = \frac{(1 + \beta_{fusion}^2) S_{consist,k} S_{contrib,k}}{\beta_{fusion}^2 S_{consist,k} + S_{contrib,k} + \varepsilon}. \quad (6)$$

The harmonic fusion requires both reference consistency and peer consistency, with $\beta_{fusion} > 1$ placing greater emphasis on the clean direction. This combination addresses different ambiguities in the two signals: agreement among clients can reflect collusion, whereas disagreement with a clean direction can arise from legitimate Non-IID data. The resulting score measures current contribution quality;

the temporal stage determines how that observation should affect continued participation.

4.2. Dual-Stream State Evolution

A low content score can reflect either a useful client's skewed data or a harmful update. Since those cases require different participation decisions, Trust Flow retains historical utility and short-term risk separately. The historical stream treats normalized content quality as soft evidence in a decayed Beta reputation model. Good evidence increases α_k , while weak evidence increases β_k ; the expected historical utility is

$$HistPerf_k^{(t)} = \frac{\alpha_k^{(t)}}{\alpha_k^{(t)} + \beta_k^{(t)} + \varepsilon}. \quad (7)$$

Decay allows later observations to revise the contribution history rather than preserving early evidence indefinitely.

The history channel changes slowly so that a few weak contributions do not erase a client's accumulated utility. For a legitimate long-tail client, low similarity can lower aggregation privilege while leaving later rounds available for recovery. The risk channel instead retains recent warnings through $RiskEMA_k^{(t)} = \beta_r RiskEMA_k^{(t-1)} + (1 - \beta_r) Risk_k^{inst}$, where $Risk_k^{inst}$ is the maximum current probe. Taking the maximum preserves a strong warning across probe channels; maintaining risk separately prevents a favorable history from altering that risk estimate. The state-transition rules then determine whether the evidence warrants review, quarantine, or permanent exclusion.

The separation changes how borderline clients are treated. A client with weak content quality but no rising risk can remain in the system with reduced influence, giving later rounds an opportunity to clarify whether the deviation was benign. A client with favorable historical utility but rising attack risk instead faces restricted aggregation privileges. Low historical utility can lead to temporary soft isolation, while the risk stream distinguishes sustained malicious behavior from a recoverable contribution deficit.

The two streams meet only when computing aggregation privilege:

$$\begin{aligned} RawScore_k^{(t)} &= (TrustScore_k^{(t)})^\alpha \\ &\times (ContentScore_k^{(t)})^\beta (HistPerf_k^{(t-1)})^\gamma \\ &\times (1 - RiskEMA_k^{(t-1)})^p. \end{aligned} \quad (8)$$

The resulting state machine distinguishes NORMAL, SUSPECT, QUARANTINE, and BLACKLIST, as shown in Fig. 4. A sleeper-burst attacker may accumulate historical utility during benign rounds, but a later trigger injection activates the risk probes and restricts its influence. A benign long-tail client follows a different path: low current similarity mainly affects its utility stream, and later positive contributions allow recovery. Keeping those paths separate avoids using one threshold for both contribution filtering and permanent exclusion.

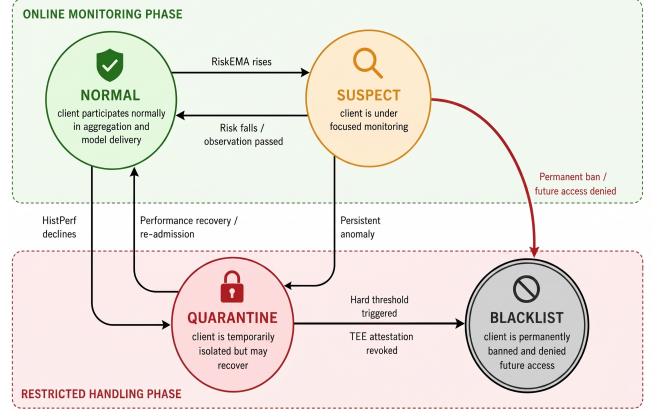


Figure 4. Client state-transition automaton driven by historical utility and short-term risk.

4.3. Risk-Gated Layered Aggregation

Backdoor effects may concentrate in deeper layers even when shallow features retain useful heterogeneous information. A uniform clipping rule then faces a tradeoff between suppressing the affected layers and preserving contributions elsewhere. Figure 5 shows how Trust Flow addresses this through layer-specific sensitivity, entry thresholds, and clipping radii. Each layer l combines privacy exposure, reference-gradient utility, and security deviation into a sensitivity score. The privacy term decreases with depth, the utility term reflects the relative reference-gradient norm of the layer, and the security term measures deviation from the reference direction. Their weighted sensitivity $S_{total}^{(l)}$ determines both the admission threshold and clipping radius:

$$\theta^{(l)} = \mu_{base} + \lambda_s S_{total}^{(l)}, \quad C^{(l)} = \frac{C_{base}}{S_{total}^{(l)} + \varepsilon_c}. \quad (9)$$

A layer showing stronger suspicious drift therefore has a higher entry threshold and a smaller clipping radius. Layers that remain close to the reference retain more capacity for benign heterogeneity.

The surviving clients are $\Phi^{(l)} = \{k \in \mathcal{A} \mid RawScore_k^{(t)} \geq \theta^{(l)}\}$. Their weights are re-normalized:

$$\begin{aligned} \tilde{w}_k^{(l)} &= \frac{RawScore_k^{(t)}}{\sum_{j \in \Phi^{(l)}} RawScore_j^{(t)} + \varepsilon}, \\ \widehat{\Delta W}_k^{(l)} &= \frac{\Delta W_k^{(l)}}{\max(1, \|\Delta W_k^{(l)}\|_2 / C^{(l)})}. \end{aligned} \quad (10)$$

The final layer update is $\Delta W_{global}^{(l)} = \sum_{k \in \Phi^{(l)}} \tilde{w}_k^{(l)} \widehat{\Delta W}_k^{(l)}$. Re-normalization keeps the effective learning rate from shrinking merely because risky clients were removed.

Layer-wise control allows the same client update to receive different treatment across the network. When a semantic backdoor mainly alters deep features, sensitivity-dependent thresholds and clipping can constrain those layers

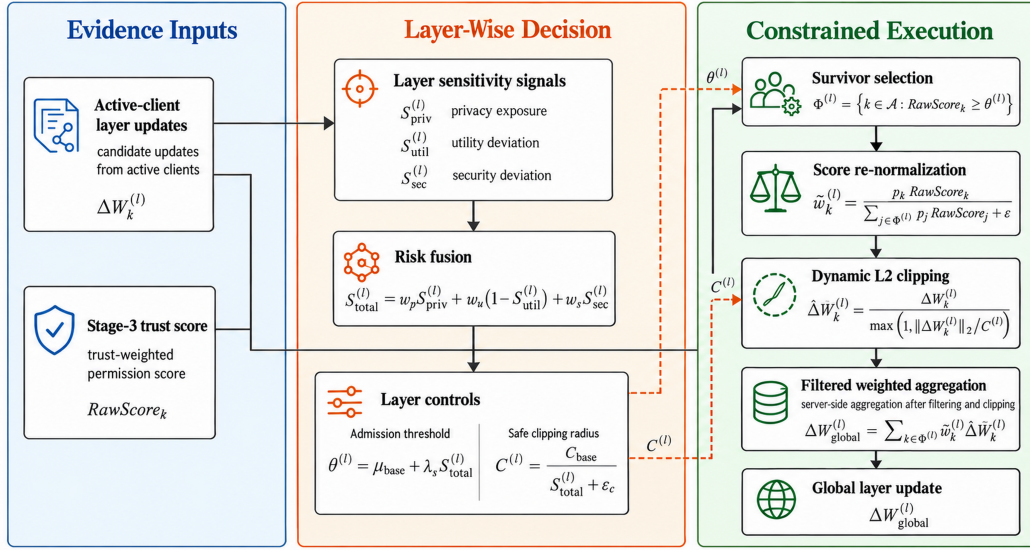


Figure 5. Risk-gated layer-wise clipping and differentiated aggregation.

while allowing more of a consistent shallow-layer update to contribute. Re-normalization addresses a separate optimization effect: removing clients also removes aggregation weight, which would otherwise reduce the update magnitude merely because filtering occurred. Projecting the surviving scores back to a valid aggregation simplex separates that weight loss from the intended clipping operation.

4.4. Analytical View

The dual-stream structure provides a way to interpret the false-positive–attack-suppression tradeoff. Suppose a legitimate long-tail client’s single-round content score has mean μ_{clean} and variance σ_{clean}^2 . A rigid threshold can reject the client when an otherwise benign observation falls below the threshold. Under the exponential-smoothing view, the steady-state variance follows the low-pass form

$$\text{Var}\left(\text{HistPerf}_k^{(\infty)}\right) = \frac{1 - \beta_h}{1 + \beta_h} \sigma_{clean}^2. \quad (11)$$

As β_h approaches one, short-term fluctuations have less influence on historical utility. The risk stream has a different role: taking the maximum across probes preserves an attack warning before it enters the EMA. Historical smoothing and risk accumulation thus address different causes of a low contribution score rather than forcing them into one reputation threshold.

4.5. Risk Probe Computation

The short-term risk stream uses intentionally redundant probes. The gradient channel measures direction deviation from g_{root} and abnormal update magnitude relative to round statistics. These checks expose sign flipping and gradient

scaling, but a clean-label attack can retain a direction close to benign updates. The clean probe therefore applies a candidate update to a small server-side probe set and records the increase in cross-entropy loss. The trigger probe compares high-level activation histograms:

$$r_{trigger,k}^{(t)} = \text{KL}(\mathcal{H}(A_{base}) \parallel \mathcal{H}(A_k)). \quad (12)$$

Here A_{base} and A_k denote the baseline and candidate-model activations. Activation drift supplies evidence of semantic or clean-label changes that may leave low-level update statistics largely unchanged.

The maximum rule combines channels whose weaknesses differ. A high-amplitude update may be exposed before it changes validation loss, whereas a stealthy backdoor may first appear as a clean-loss increase or an activation shift. Peer disagreement remains supporting evidence because benign long-tail updates can differ from the majority. These channels make the source of a warning visible to the temporal stage without requiring every probe to agree.

4.6. Assurance Composition and Reference Role

The evidence sources complement one another because each resolves a different uncertainty. Attestation and privacy-aware verification establish execution provenance and bounded statistics for admission, while cloud auditing examines the submitted update against reference and peer evidence. Temporal states retain the distinction between a transient deviation and persistent risk. Passing the measured execution path therefore leads into semantic auditing; it does not bypass it. Likewise, low similarity can reduce contribution without serving as sufficient evidence for permanent exclusion.

A clean validation gradient supplies a stable reference when available, while the peer-reference branch constructs

a guiding direction from admitted updates when it is unavailable. Reference agreement contributes to content quality alongside peer consistency; historical contribution and recent risk then influence participation. The reference is therefore one input to auditing rather than a substitute for the dual-stream policy.

4.7. From Separate States to Participation Decisions

The distinction between historical contribution and recent risk matters most for clients near a participation boundary. A client with low utility but low risk may receive reduced influence or reversible review, allowing later evidence to clarify whether its deviation was benign. A client with substantial past utility and rising risk instead faces restrictions despite that history. Persistent high risk is required for irreversible blacklisting, so the same threshold need not serve simultaneously as a contribution filter and a permanent-exclusion rule.

For fixed admission factor, content score, and historical utility, write $B_k = (TrustScore_k)^\alpha (ContentScore_k)^\beta (HistPerf_k)^\gamma$. With $p > 0$ and $0 \leq r < 1$, the score satisfies

$$\frac{\partial [B_k(1-r)^p]}{\partial r} = -pB_k(1-r)^{p-1} \leq 0, \quad (13)$$

$$r = RiskEMA_k^{(t-1)}.$$

At $r = 1$, the score is zero. This statement concerns the unnormalized score with the other factors fixed; layer selection and normalization determine the update that is eventually applied. SUSPECT and QUARANTINE are recoverable states, allowing later evidence to revise a temporary restriction. Low similarity alone is therefore not interpreted as maliciousness.

The mechanism-level evidence is separated across the ablations. The reputation ablation compares single-stream variants, HistPerf-only, and the complete dual stream on false bans and detection. The component ablations isolate probe contributions and re-normalization. The first comparison tests the temporal policy; the second tests the evidence supplied to it and the optimization behavior after filtering.

5. Experiments

5.1. Setup

Table 1 summarizes the experimental configuration. Experiments use Flower 1.5.0 and PyTorch on CIFAR-10 with a lightweight CNN. The testbed is an Inspur server with dual CPUs and five NVIDIA A10 GPUs, running Ubuntu 20.04.6 LTS and CUDA 12.8. Intel TDX supplies the client execution boundary described above. The 50,000 training images are distributed among 20 admitted clients. A shared, balanced pool of 5,000 samples provides a common alignment basis; clients 6–11 follow a Dirichlet partition with $\alpha = 1.0$, and clients 12–19 use a strong long-tail partition

with $\alpha = 0.1$. Aggregate results are computed over five independent runs with different random seeds, while client-state trajectories use one representative fixed-seed run.

TABLE 1. EXPERIMENTAL CONFIGURATION AND MAIN HYPERPARAMETERS.

Category	Setting	Value
FL task	Dataset/model/admitted clients	CIFAR-10/light CNN/ $K = 20$
Optimization	Local epochs, lr, momentum, rounds	3, 0.001, 0.9, 30
Data heterogeneity	Shared pool; clients	5000; $\alpha = 1.0 / 0.1$
Trust streams	$\alpha_1, \alpha_2, \beta_{fusion}; \beta_h, \beta_r$	0.6, 0.4, 2.0; 0.9, 0.85
Layer gating	$\alpha, \beta, \gamma, p; \mu_{base}, \lambda_s$	3.0, 1.0, 0.5, 1.0; 0.4, 1.2

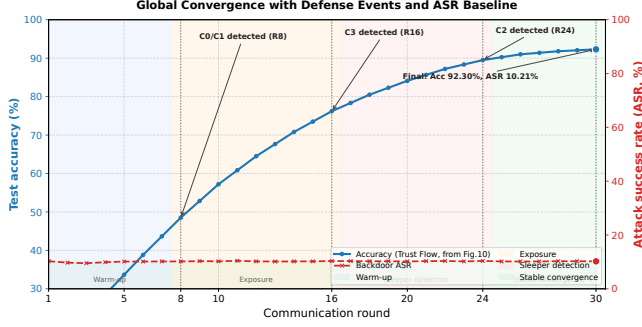
The formal 30% attack setting uses six malicious clients. Client 0 performs label flipping with $\gamma = 0.5$; Client 1 injects a trigger backdoor into 20% of samples and targets class 0; Client 2 uses clean-label backdoor perturbations; Client 3 applies semantic backdoor bias; Client 4 performs sign flipping; and Client 5 scales updates by a factor of 100. Five Sybil nodes without valid certificates and three free-rider nodes with forged low workload validate Phase-One gating. These eight nodes are intercepted before statistical aggregation, so subsequent metrics concern the 20 admitted clients. Each run uses 30 communication rounds and three local epochs.

Accuracy and ASR are reported as means and standard deviations over five independent runs. Permanent-ban FPR counts benign clients that are irreversibly blacklisted; temporary SUSPECT and QUARANTINE states are recoverable controls. Stateless baselines have no comparable permanent-ban state, so their FPR is reported as N/A. The FedAvg, Krum, Trimmed Mean, FLTrust, and Trust Flow comparison uses the same node partitioning, attack proportion, optimizer configuration, communication budget, and Non-IID split. Fixed-seed trajectories illustrate client-level transitions, while aggregate tables summarize repeated runs.

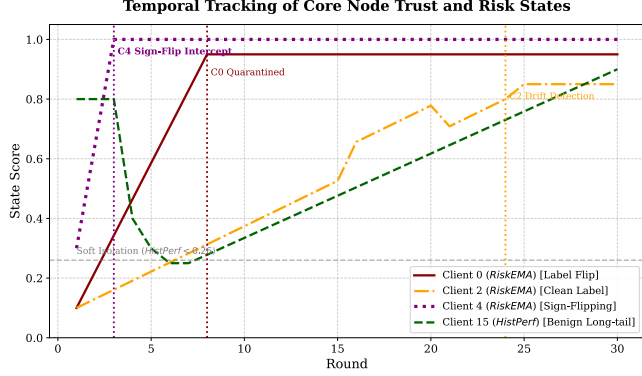
5.2. Overall Defense Performance

Figure 6 collects convergence, ASR, client-state trajectories, and feature separation. Under the 30% mixed-attack setting, Trust Flow reaches 92.31% accuracy and 10.21% ASR. Explicit poisoning and parameter attacks are detected early, while stealthier clean-label drift requires persistent probe evidence. A legitimate long-tail client may enter SUSPECT or QUARANTINE temporarily without accumulating the sustained risk associated with permanent exclusion. The trajectories connect this distinction to the aggregate accuracy and attack-suppression results.

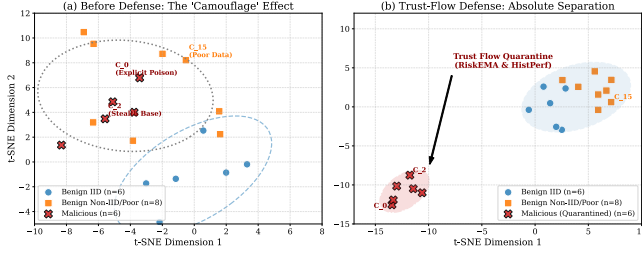
The trajectories in Fig. 6(b) connect the aggregate results to client-level decisions. Elevated risk is retained for label, trigger, semantic, sign, and scaling attacks, whereas a benign long-tail client’s weak contribution need not produce the sustained risk required for blacklisting. Later positive contributions allow its historical utility to recover. Figure 6(c) visualizes the resulting trust-state features through



(a) Accuracy and ASR convergence.



(b) Node trust and risk trajectories.



(c) t-SNE separation before/after defense.

Figure 6. Main defense behavior under mixed attacks.

t-SNE; accuracy, ASR, and state-transition metrics provide the quantitative basis for assessing the defense.

In the matched classical comparison in Table 2, Krum reaches 64.20% accuracy, Trimmed Mean 71.35%, and FLTrust 81.25%, while Trust Flow reaches 92.31%. These values do not establish a universal ranking. DeepSight, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA require attack-specific or privacy-preserving interfaces that are not reproduced here, so their absence is a comparison limitation. The 50% result remains an out-of-assumption stress test rather than evidence of a theoretical Byzantine boundary.

5.3. Ablation, Sensitivity, and Overhead

Table 3 examines whether one state can serve both retention and detection. The aggressive single-stream variant attains high recall at the cost of more permanent false positives, while the conservative variant reduces false positives

but misses more malicious clients. HistPerf-only further illustrates the limitation of relying on contribution history: it retains benign clients but provides little attack sensitivity. The dual-stream result supports assigning those roles to separate states. The reported TrustReport size is about 15 KB, and server aggregation plus audit increases from 2.10 s to 4.85 s. Optional cryptographic secure aggregation remains outside the stated system boundary. The component and heterogeneity results are shown in Figs. 7 and 8.

The server timing covers aggregation and content audit. The measured auxiliary security-path overhead is approximately 11 ms per admitted-client round. Device-side TEE Quote generation, signing, and lightweight monitoring are measured separately at approximately 12–15 ms per admitted-client round. These values describe their respective measurement scopes rather than one end-to-end deployment latency. CPU-side TDX isolation does not by itself establish protection of GPU execution or GPU memory.

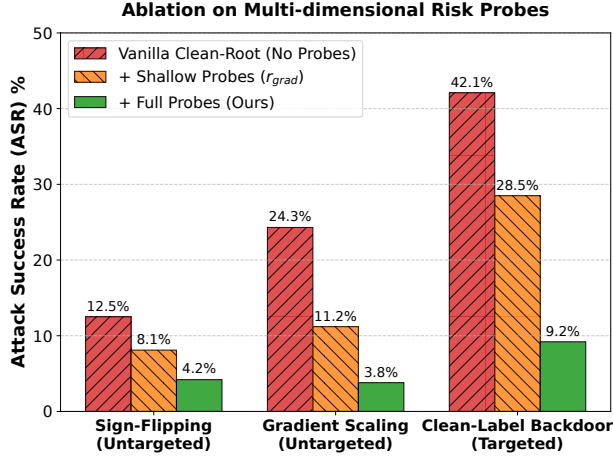
The FPR/TPR values in Table 3 use the final-ban definition. Counting SUSPECT as a recoverable review state gives 88.33% per-round malicious-client coverage and a 14.79% benign-client review rate, with no permanent false ban. Figure 7 separates the effects of probe coverage and re-normalization: clean-reference and shallow checks handle explicit parameter attacks, while activation and clean-loss probes reduce the reported clean-label ASR to 9.2%. Survivor re-normalization avoids an implicit learning-rate decay after filtering. Figure 8 shows that Trust Flow remains near 92% accuracy at $\alpha = 0.1$.

The Dirichlet sweep in Fig. 8 holds label space, model, optimizer, and attack protocol fixed while varying data heterogeneity. As α decreases, benign updates become less aligned, making the filtering decisions of Krum and FLTrust more likely to reject legitimate drift. Trust Flow is less sensitive in the reported sweep because current content quality is considered together with historical utility and recent risk. The default $\beta_{fusion} = 2.0$ gives the best observed accuracy–ASR balance. The reported 2.10 s to 4.85 s runtime covers server-side aggregation and auditing, not end-to-end TDX deployment latency.

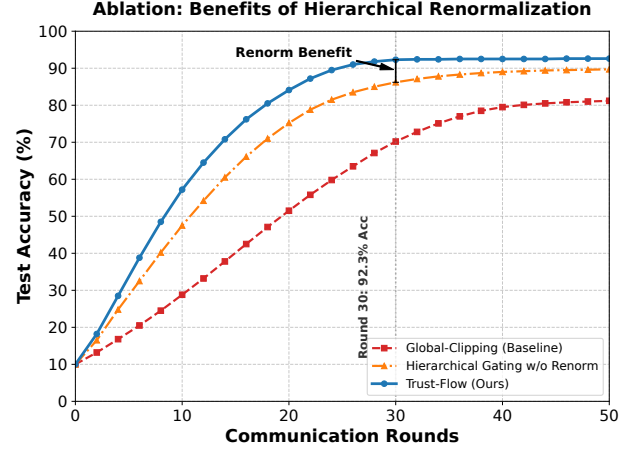
TABLE 2. PERFORMANCE COMPARISON UNDER 30% MIXED ATTACKS AND BOUNDARY STRESS TESTING.

Method/Setting	Core mechanism	Acc.	ASR	Perm. FPR [†]
FedAvg [2]	no defense	86.15 $\pm$ 1.20	92.34 $\pm$ 5.10	N/A
Krum [3]	Euclidean selection	64.20 $\pm$ 2.80	12.15 $\pm$ 0.90	N/A
Trimmed Mean [4]	coordinate trimming	71.35 $\pm$ 2.40	38.60 $\pm$ 4.20	N/A
FLTrust [5]	clean anchor weighting	81.25 $\pm$ 1.50	15.42 $\pm$ 1.10	N/A
FLAME [9]	clustering, clipping, noise	73.18 $\pm$ 1.80	19.00 $\pm$ 0.60	N/A
FLDetector [7]	update consistency	65.89 $\pm$ 2.30	23.00 $\pm$ 1.20	N/A
FreqFed [22]	frequency analysis	79.37 $\pm$ 0.90	14.00 $\pm$ 1.10	N/A
Trust Flow	dual stream + layered gating	92.31 $\pm$ 0.09	10.21 $\pm$ 0.05	0.00
Boundary stress: 50% malicious	10/20 malicious	91.81 $\pm$ 0.10	10.46 $\pm$ 0.06	0.00

[†] Permanent-ban FPR is defined only for Trust Flow and its stateful ablations. Krum, Trimmed Mean, and FLTrust are stateless in this comparison and are therefore marked N/A. Temporary SUSPECT/QUARANTINE states are recoverable and reported separately through trajectories.



(a) Multi-dimensional risk-probe ablation.



(b) Weight re-normalization effect.

Figure 7. Component-level ablation of risk probes and re-normalization.

TABLE 3. ABLATION AND OVERHEAD SUMMARY.

Study	Variant / architecture	FPR or overhead	TPR / time
Reputation	Single-stream EMA, aggressive	18.25% FPR	95.12% TPR
Reputation	Single-stream EMA, conservative	2.10% FPR	48.33% TPR
Reputation	HistPerf-only	0.15% FPR	21.05% TPR
Reputation	Dual-stream Trust Flow	0.00% FPR	100.00% TPR
Overhead	Standard FedAvg	0 KB extra	2.10 s aggregation
Overhead	Trust Flow	~15 KB TrustReport	4.85 s incl. audit

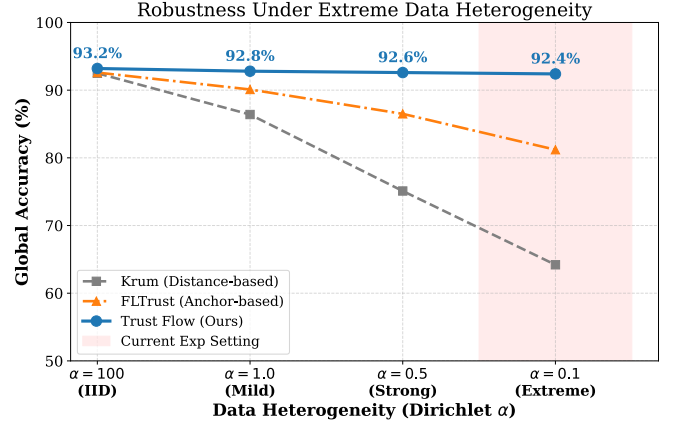


Figure 8. Robustness stress test under varying data heterogeneity.

6. Discussion and Conclusion

Trust Flow connects execution admission, privacy-aware statistical checks, content auditing, and layer-wise aggregation through separate historical-utility and short-term-risk states. This separation addresses the tension between retaining benign long-tail contributions and restricting harmful

updates from clients with favorable histories. Execution and current-update evidence inform the states, while persistence rules distinguish recoverable restrictions from permanent exclusion. In the reported mixed-attack experiments, this separation keeps permanent-ban FPR near zero while retaining high recall against the tested mixed attacks. The results

relate the participation policy to both attack suppression and benign-client retention.

The evaluated profile is server-assisted vision FL with a small trusted reference partition and an attestation-capable client path. The peer-reference branch provides a guiding direction when a clean validation gradient is unavailable. Microarchitectural side channels and confidential-VM resource contention remain outside the threat model. The probes are tailored to vision classification; report size and server audit overhead are kept separate from end-to-end latency. Cryptographic secure aggregation is outside the main boundary because the server must inspect individual evidence. This profile states where the trust-flow contract applies and provides a basis for extending it to other data modalities and deployment conditions.

References

- [1] P. Kairouz, H. B. McMahan, B. Avent *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agueray Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, vol. 54, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [3] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine learning with adversaries: Byzantine tolerant gradient descent,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://papers.nips.cc/paper/6617-machine-learning-with-adversaries-byzantine-tolerant-gradient-descent>
- [4] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, 2018, pp. 5650–5659. [Online]. Available: <https://proceedings.mlr.press/v80/yin18a.html>
- [5] X. Cao, M. Fang, J. Liu, and N. Z. Gong, “FLTrust: Byzantine-robust federated learning via trust bootstrapping,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2021.
- [6] C. Fung, C. J. M. Yoon, and I. Beschastnikh, “The limitations of federated learning in Sybil settings,” in *Proceedings of the 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2020, pp. 301–316. [Online]. Available: <https://www.usenix.org/conference/raid2020/presentation/fung>
- [7] Z. Zhang, X. Cao, J. Jia, and N. Z. Gong, “FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients,” in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2545–2555.
- [8] P. Rieger, T. D. Nguyen, M. Miettinen, and A. R. Sadeghi, “Deep-Sight: Mitigating backdoor attacks in federated learning through deep model inspection,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.
- [9] T. D. Nguyen, P. Rieger, H. Chen, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen, A. Mirhoseini, S. Zeitouni, F. Koushanfar, A.-R. Sadeghi, and T. Schneider, “FLAME: Taming backdoors in federated learning,” in *Proceedings of the 31st USENIX Security Symposium*, 2022, pp. 1415–1432. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/nguyen>
- [10] P. Rieger, T. Krauß, M. Miettinen, A. Dmitrienko, and A. R. Sadeghi, “CrowdGuard: Federated backdoor detection in federated learning,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2024.
- [11] H. Yang, W. Xi, Y. Shen, C. Wu, and J. Zhao, “RoseAgg: Robust defense against targeted collusion attacks in federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2951–2966, 2024.
- [12] Y. Chen, F. Luo, T. Li, T. Xiang, Z. Liu, and J. Li, “A training-integrity privacy-preserving federated learning scheme with trusted execution environment,” *Information Sciences*, vol. 522, pp. 69–79, 2020.
- [13] T. Xu, K. Zhu, A. Andrzejak, and L. Zhang, “Distributed learning in trusted execution environment: A case study of federated learning in SGX,” in *Proceedings of the 7th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC)*, 2021, pp. 450–454.
- [14] S. Queyruat, V. Schiavoni, and P. Felber, “Mitigating adversarial attacks in federated learning with trusted execution environments,” in *Proceedings of the IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*, 2023, pp. 626–637.
- [15] Z. Lu, L. Wang, Z. Zhang, M. Huang, J. Wang, and M. Li, “TMT-FL: Enabling trustworthy model training of federated learning with malicious participants,” *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 3, pp. 2723–2740, 2025.
- [16] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How To Backdoor Federated Learning,” in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, vol. 108, 2020, pp. 2938–2948. [Online]. Available: <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
- [17] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao, “Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks,” in *Proceedings of the IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 707–723.
- [18] J. Zhang, C. Zhu, X. Sun, C. Ge, B. Chen, W. Susilo, and S. Yu, “FLPurifier: Backdoor defense in federated learning via decoupled contrastive training,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4752–4766, 2024.
- [19] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, “ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1639–1654, 2022.
- [20] L. Wang, M. Huang, Z. Zhang, M. Li, J. Wang, and K. Gai, “RaSA: Robust and adaptive secure aggregation for edge-assisted hierarchical federated learning,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 4280–4295, 2025.
- [21] X. Zhang, Z. Chen, G. Chen, X. Feng, Q. Shen, and Z. Wu, “RPPFL: Robust and privacy-preserving federated learning via trusted execution environments,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.
- [22] H. Fereidooni, A. Pegoraro, P. Rieger, A. Dmitrienko, and A.-R. Sadeghi, “FreqFed: A frequency analysis-based approach for mitigating poisoning attacks in federated learning,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2024. [Online]. Available: <https://doi.org/10.14722/ndss.2024.23620>